Comparativa de consumo de recursos computacionales entre algoritmos clásicos y sus equivalentes cuánticos simulados en Qiskit

Comparative Analysis of Computational Resource Consumption Between Classical Algorithms and Their Quantum Equivalents Simulated in Qiskit

Moreno Solano Justin Thomas[1]
Universidad Ean, Facultad de Ingeniería, Bogotá, Colombia
Correspondencia: jmoreno63009@universidadean.edu.co

Resumen: Los circuitos cuánticos se simulan de forma habitual en computadoras clásicas, pero el costo real no se deduce de la complejidad teórica de los algoritmos. En este trabajo, se analiza cómo varía el tiempo de ejecución y el uso de memoria cuando se simulan algoritmos cuánticos en Qiskit Aer, en comparación con sus equivalentes clásicos, a medida que el tamaño del problema aumenta. Se utilizó un enfoque cuantitativo, con un diseño experimental, para comparar cuatro pares de algoritmos (búsqueda no estructurada, Deutsch-Jozsa, transformada de Fourier y factorización) en tamaños de 1 a 32, con tres repeticiones, en un procesador (CPU) y en una unidad de procesamiento gráfico (GPU), en un portátil con 15.34 GiB de memoria RAM y 6 GB de memoria de video. Se analizaron 987 mediciones válidas. El tiempo de ejecución de la simulación por vector de estado creció exponencialmente y fue de 3 a 3,221 veces mayor que el tiempo de ejecución de la versión clásica en Grover y en la transformada de Fourier. En cambio, Deutsch-Jozsa, simulado con el método estabilizador, fue más rápido a partir de 12 bits. La memoria utilizada por la simulación por vector de estado fue exactamente de $16 * 2^n$ bytes más una base fija, lo que limitó la simulación a 29 qubits en CPU y 28 en GPU. La GPU aceleró la ejecución hasta 3,3 veces en los tamaños más grandes y la variación entre repeticiones fue de 5 % (mediana) en las ejecuciones de 1 s o más. De acuerdo con los resultados, el costo de simular depende del método de simulación y de la implementación, y no solo del número de qubits.

Abstract: Quantum circuits are routinely simulated on conventional computers, but the actual cost cannot be inferred from the theoretical complexity of the algorithms. This study evaluates how execution time and memory consumption vary when quantum algorithms are simulated in Qiskit Aer, compared with their classical equivalents, as problem size increases. A quantitative, experimental design was adopted: four algorithm pairs (unstructured search, Deutsch-Jozsa, Fourier transform and factoring) were compared for sizes from 1 to 32, with three repetitions and simulation on the central processing unit (CPU) and the graphics processing unit (GPU), on a laptop with 15.34 GiB of RAM and 6 GB of video memory. A total of 987 valid measurements were analyzed. State-vector simulation time grew exponentially and was 3 to 3,221 times that of the classical version for Grover's algorithm and the Fourier transform, whereas Deutsch-Jozsa, simulated with the stabilizer method, was faster from 12 bits onward. Memory followed exactly $16 * 2^n$ bytes plus a fixed baseline, limiting simulation to 29 qubits on CPU and 28 on GPU. The GPU sped up execution by up to 3.3 times at the largest sizes, and the variation across repetitions was 5% (median) for runs of 1 s or longer. It is concluded that the cost of simulation depends on the simulation method and the implementation, and not only

on the number of qubits.

Palabras clave: simulación cuántica; Qiskit Aer; consumo de recursos; algoritmo de Grover; aceleración por GPU

Keywords: quantum simulation; Qiskit Aer; resource consumption; Grover's algorithm; GPU acceleration

I. INTRODUCCIÓN

La simulación clásica de circuitos cuánticos se realiza para comprobar, ajustar y contrastar dispositivos y algoritmos. El tamaño del circuito que se puede simular varía en función de los recursos disponibles (Dispositivo de ejecución), de la representación matemática y de la estructura del circuito. Por ejemplo, en un supercomputador con 8,192 nodos y 0,5 petabytes de memoria se logro simular un circuito de 45 qubits. No se debe esperar que un equipo personal simule un circuito de este tamaño (Häner & Steiger, 2017). En la simulación por vector de estado, la memoria se duplica con cada qubit adicional (IBM Quantum, s. f.). Además, hay circuitos que se simulan de forma eficiente (Aaronson & Gottesman, 2004; Vidal, 2003) y la aceleración con GPU depende del método, del tamaño y de la plataforma.

Los estudios teóricos cuentan las consultas que un algoritmo hace a un oráculo; Grover (1996), por ejemplo, las reduce de un orden N a un orden √N. Esa ventaja no indica cuánta memoria ni cuánto tiempo exige simular el circuito, y la aceleración de hasta 14 veces que Faj et al. (2023) informaron con una GPU A100 no se puede extrapolar a otros equipos. Las evaluaciones comparativas muestran que el rendimiento cambia según el simulador, el circuito y la configuración (Cerrudo-Herrera et al., 2026), pero hay poca evidencia reproducible que compare, en un mismo equipo, algoritmos clásicos y sus equivalentes simulados.

Medir directamente el tiempo y la memoria permite cerrar la brecha entre la complejidad teórica y el costo real. Los datos reproducibles obtenidos en un portátil convencional sirven a estudiantes y desarrolladores que van a experimentar con Qiskit Aer sin acceso a un supercomputador.

El objetivo de este artículo es evaluar y comparar el consumo de recursos computacionales (tiempo de ejecución, memoria RAM , uso de CPU Y GPU) entre algoritmos clásicos y sus contrapartes cuánticas simuladas en Qiskit Aer. La pregunta que se pretende responder es: ¿cómo varía el consumo de recursos al simular algoritmos cuánticos, en comparación con sus equivalentes clásicos, a medida que el tamaño del problema aumenta? Para responder a esta pregunta, se seleccionaron y se implementaron cuatro pares de algoritmos, se instrumentó su medición y se analizaron los resultados por tamaño, método y dispositivo.

II. Marco Teórico y Conceptual

A. Simulación clásica de circuitos cuánticos

Un circuito cuántico transforma qubits mediante puertas y finaliza con una medición. La descripción por vector de estado del circuito contiene 2^n amplitudes complejas. Con doble precisión (16 bytes por amplitud), la descripción ocupa $16 * 2^n$ bytes. Esta cantidad no incluye la memoria temporal y los sobrecostos del proceso, Qiskit permite configurar un límite de memoria (IBM Quantum, s. f.). Esta dependencia exponencial es el límite principal de la escalabilidad de la representación (Vallero et al., 2025). Sin embargo, el costo depende del método. Los circuitos estabilizadores, que están formados por puertas CNOT, Hadamard y de fase, se simulan eficientemente (Aaronson & Gottesman, 2004). Los estados con poco entrelazamiento permiten utilizar recursos que son lineales en el número de qubits (Vidal, 2003). La contracción de redes tensoriales relaciona el costo con las propiedades del circuito, como su treewidth (Markov & Shi, 2008). Qiskit ofrece diferentes métodos de simulación (vector de estado, estabilizador y producto de estados matriciales). La GPU solo se puede utilizar en algunos de los métodos de simulación (IBM Quantum, s. f.). Por lo tanto, el número de qubits no es suficiente para describir una ejecución: también se debe documentar el método de simulación, el dispositivo y la precisión.

B. Complejidad teórica y costo de simulación

La complejidad algorítmica compara cómo se incrementan las consultas, las puertas, el tiempo o el espacio con el tamaño de la entrada. En la búsqueda no estructurada, el oráculo es la parte del algoritmo que determina si un candidato es la solución. Por ejemplo, ¿el elemento de la posición i es igual a 42? Cada comprobación se cuenta como una consulta: la búsqueda clásica la realiza hasta N veces y Grover la realiza un orden de √N veces. Esta es una mejora cuadrática y no exponencial (Grover, 1996). Esta ventaja cuenta las consultas, no el tiempo ni la memoria que se requiere para simular el circuito en un procesador clásico, y la comparación solo es justa si el costo del oráculo se incluye en ambas versiones. Por lo tanto, es importante diferenciar tres niveles: la complejidad teórica, la ejecución física en un dispositivo cuántico y la simulación clásica. Este trabajo se encuentra en el tercer nivel.

La aceleración por GPU tampoco es una característica del algoritmo: depende del tamaño del circuito, de la implementación y del movimiento de datos. En las configuraciones con múltiples GPU, la aceleración está limitada por la transferencia de datos (Faj et al., 2023).

C. Variables de análisis

El estudio cambia la implementación (clásica o cuántica simulada), el tamaño de la entrada, el método de simulación y el dispositivo, y mide el tiempo de ejecución, el máximo de memoria RAM que utiliza el proceso y el uso de la GPU. Los parámetros de la ejecución se fijan o se registran; la Figura 1 muestra cómo se relacionan las variables y la Tabla 1 detalla su medición. La unidad de análisis es una ejecución individual que se mide en un proceso aislado.

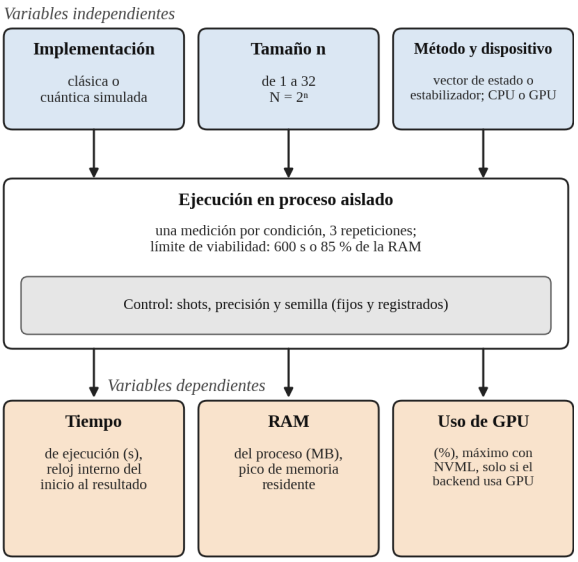


Figura 1. Modelo conceptual: variables independientes, de control y dependientes.

Nota. Elaboración propia.

Tabla 1. Variables del estudio y su medición

Variable	Rol	Medición	Unidad
Implementación	Independiente	Versión clásica o cuántica simulada de cada problema	Categórica
Tamaño (n)	Independiente	De 1 a 32 en pasos de 1; $N = 2^n$	Qubits o bits
Método y dispositivo	Independiente / control	Método de Aer y CPU o GPU; se verifican en el resultado de Qiskit	Categórica
Tiempo de ejecución	Dependiente	Reloj del monitor interno, desde el inicio del trabajo hasta el resultado	s
RAM del proceso	Dependiente	Pico de la memoria residente, muestreada cada 0,1 s por el proceso padre	MB
Utilización de GPU	Dependiente	Máximo muestreado con NVML, solo si el backend usa GPU	%
Parámetros de corrida	Control	Shots, precisión y semilla, fijados y registrados	—

Nota. NVML: biblioteca de NVIDIA para consultar el estado de la GPU.

III. Metodología

A. Enfoque, alcance y diseño

El estudio se realizó con un enfoque cuantitativo, debido a que la pregunta requiere que el consumo se exprese como una cantidad numérica con unidades y que se describa su crecimiento. El fenómeno (el consumo de recursos de un proceso) no depende de las opiniones de las personas. El alcance del estudio es descriptivo comparativo, y tiene un elemento explicativo en un sentido operativo: se describe el consumo de cada implementación, se compara la versión clásica con la versión simulada utilizando el mismo criterio de éxito, y se explican las diferencias como resultado de los factores que se manipularon en un ambiente controlado. El diseño del estudio es un diseño experimental computacional de medidas repetidas: se manipulan la implementación, el tamaño, el método, en el mismo dispositivo y se fijan o se registran las demás condiciones. Cada medición es una ejecución en un proceso aislado. En el estudio no participaron personas y no se recopilaron datos personales. El estudio no tiene la intención de demostrar que el hardware cuántico físico

tiene alguna ventaja, ni tampoco se pretende generalizar los resultados a todos los algoritmos o a todos los simuladores.

B. Algoritmos evaluados

Los algoritmos que se evaluaron se seleccionaron mediante un muestreo intencional. Se eligieron pares de algoritmos que tienen un equivalente cuántico conocido, una entrada, una salida y un criterio de éxito bien definidos, un tamaño que se puede controlar y una implementación en Qiskit (Tabla 2).

Tabla 2. Pares de algoritmos que se evaluaron

Problema	Versión clásica	Versión cuántica simulada	Tamaño n
Búsqueda no estructurada	Búsqueda lineal en una lista de 2^n elementos	Algoritmo de Grover	n qubits ($N = 2^n$)
Función constante o balanceada	Evaluación exhaustiva de las 2^n entradas	Deutsch-Jozsa	n bits (n + 1 qubits)
Transformada de Fourier	FFT de NumPy sobre 2^n valores	QFT	n qubits
Factorización	División de prueba de un número de n bits	Shor con N = 15	n qubits de conteo (+ 4)

Nota. *FFT: transformada rápida de Fourier; QFT: transformada cuántica de Fourier.*

C. Equipo y condiciones

Las mediciones se realizaron en un portátil Lenovo LOQ 15IRH8 que tiene un procesador Intel Core i5-12450H, 15,34 GiB de memoria RAM y una GPU NVIDIA GeForce RTX 3050 Laptop con 6 GB de memoria de video. Con un sistema operativo linux (Omarchy 3.8.5) y los lenguajes de programación utilizados fueron Python 3.12, Qiskit 1.4.5 y qiskit-aer-gpu 0.15.1. Grover y la transformada cuántica de Fourier se simularon con el método de vector de estado, y Deutsch-Jozsa y Shor con el método automático de Qiskit Aer. La GPU solo se utilizó en la parte cuántica. Se evaluaron tamaños de 1 a 32 en pasos de 1, y se realizaron tres repeticiones por condición y 1,024 shots (2,048 en Shor). El diseño del estudio contemplaba la posibilidad de realizar un máximo de 1,152 ejecuciones (4 pares × 32 tamaños × 3 repeticiones × 3 series: clásica, CPU y GPU).

D. Instrumentos y procedimiento

Un programa en Python ejecutó cada medición en un proceso aislado. En la versión cuántica, antes de realizar la medición se ejecutó el algoritmo una vez con un tamaño pequeño para comprobar el funcionamiento del sistema. Esta ejecución no se registró. El programa en Python cambió el orden de las series en cada repetición de forma aleatoria. Cada proceso registró su tiempo de ejecución con un monitor interno (psutil y NVML) y el proceso padre tomó una muestra de la memoria residente cada 0,1 s para obtener el pico de la RAM. Un sistema de seguridad interrumpió la ejecución de cualquier programa que tomara más de 600 s o que usara más del 85 % de la RAM total. La ejecución de cada serie (algoritmo, versión y dispositivo) se detuvo cuando se produjo el primer fallo por falta de recursos. Ese punto se reportó como el límite de viabilidad de la serie. El tiempo de ejecución se resumió con la mediana de las tres repeticiones, y la variación del tiempo de ejecución se resumió con la diferencia entre el máximo y el mínimo

tiempo de ejecución dividida por la mediana del tiempo de ejecución. El rango es solo una guía, ya que solo se realizaron tres repeticiones.

E. Búsqueda bibliográfica

La literatura del marco teórico se buscó con Elicit el 30 de septiempre de 2026, con la consulta “Ayudame a encontrar documentos que sigan la siguiente ecuacion booleana ("quantum algorithm" OR "quantum circuit") AND ("classical simulation" OR "statevector" OR "Qiskit Aer") AND ("resource consumption" OR "execution time" OR "memory limit" OR "GPU acceleration") para la redaccion del marco teorico de mi documento de investigacion”, sobre la base de datos Semantic Scholar y sin aplicar filtros restrictivos de año, idioma o tipo de documento.

IV. Resultados

A. Características de la implementación

Un total de 1.032 ejecuciones fueron registradas para una duración de 3 horas 53 minutos. Un resultado fue obtenido por 987 ejecuciones, y 45 ejecuciones fallaron por superar el límite de viabilidad, es decir, por superar el límite de tiempo o el de consumo de RAM. Las ejecuciones que fallaron por superar el límite de viabilidad se dividieron en 15 ejecuciones que fueron detenidas por superar el tiempo límite de ejecución o por superar el límite de la RAM, y 30 ejecuciones que fallaron por un error: 6 ejecuciones de Grover con n = 1, 18 ejecuciones que necesitaban circuitos de más de 29 qubits y 6 ejecuciones que fallaron por falta de memoria de la GPU. La RAM se mide con el pico de la RAM que fue medido por el proceso padre. Esto sucede ya que el monitor interno solo tomó 1 o 2 muestras en la mayoría de las ejecuciones de QFT y de Shor. No se tiene una medición válida de la CPU, ni de la memoria del disco, ni de la memoria de la GPU. No se registró si los resultados eran correctos en las ejecuciones de Grover, de Deutsch-Jozsa ni de QFT. En Shor, el circuito cuántico factoriza siempre N = 15. Por lo tanto, no se puede hacer una comparación punto a punto con la versión clásica.

B. Tiempo de ejecución

En Grover, el tiempo de ejecución de la búsqueda lineal esta cerca de los 0,6 ms hasta que n = 12. A partir de n = 12, el tiempo de ejecución de la búsqueda lineal se duplica por cada qubit que se añade, hasta que alcanza unos 5,5 s con n = 28. La simulación es más lenta a medida que se aumentan los tamaños de circuito (Figura 2): el cociente entre el tiempo de ejecución de la simulación y el de la versión clásica crece unas 52 veces (n = 10) a 635 (n = 20) y a 3,221 (n = 23). Por cada qubit que se añade, el tiempo de ejecución de la simulación se multiplica por 3,2 en la CPU y por 2,6 en la GPU. En Deutsch-Jozsa, ocurre lo contrario: el tiempo de ejecución de la evaluación exhaustiva se duplica con cada bit que se añade (2,04) y llega a unos 427 s con n = 27. En cambio, el tiempo de ejecución de la simulación se mantiene entre 0,005 y 0,02 s hasta que n = 32. El tiempo de ejecución de la simulación es menor que el tiempo de ejecución de la versión clásica a partir de n = 12. Con n = 27, el tiempo de ejecución de la simulación es unas 26,000 veces menor que el tiempo de ejecución de la versión clásica.

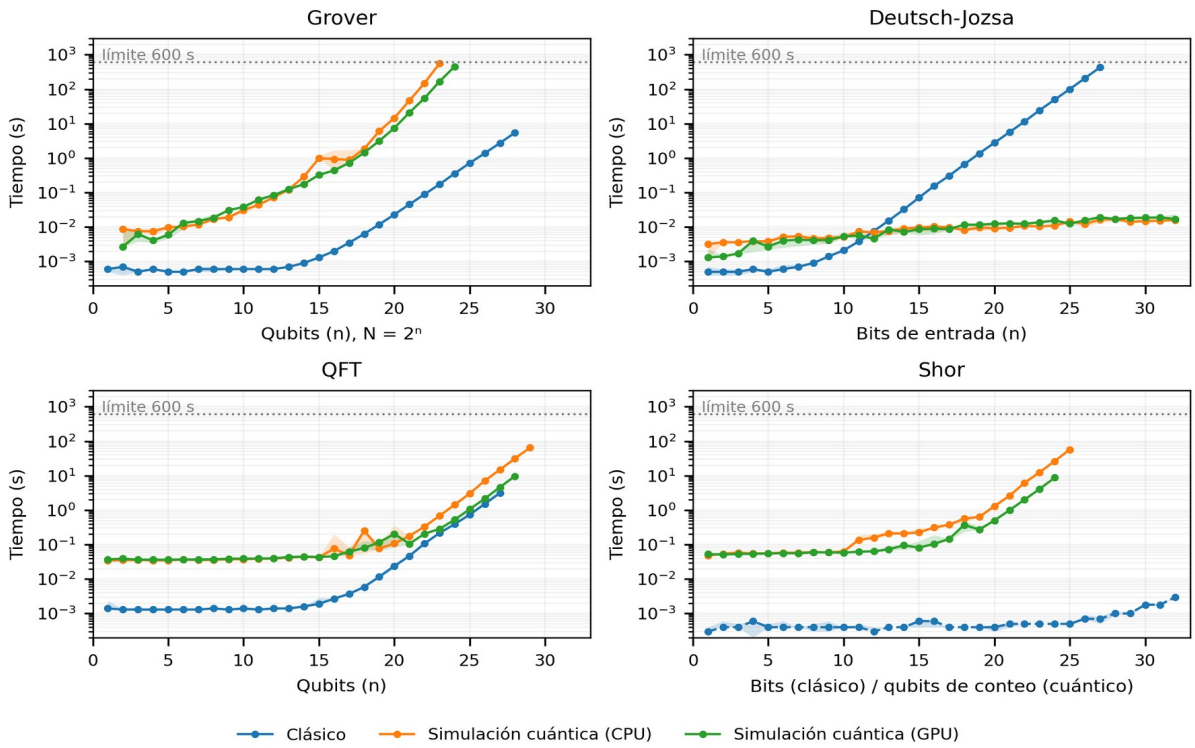


Figura 2. Tiempo de ejecución de cada algoritmo, versión y tamaño. La línea es la mediana de las 3 repeticiones; la banda es el rango entre el mínimo y el máximo tiempo de ejecución; el eje de las ordenadas está en escala logarítmica. En Deutsch-Jozsa, la serie de la GPU es igual a la serie de la CPU (el método estabilizador no utiliza la GPU). En Shor, los ejes de las abscisas son diferentes: en la versión clásica, el eje de las abscisas es el número de bits del número que se va a factorizar, y en la versión cuántica, el eje de las abscisas es el número de qubits de conteo.

Nota. Elaboración propia.

En la transformada de Fourier, la simulación tiene un tiempo de ejecución fijo de 35 a 40 ms (entre 27 y 28 veces mayor que el tiempo de ejecución de la versión clásica para $n \leq 16$). A partir de $n = 20$, tanto el tiempo de ejecución de la simulación como el tiempo de ejecución de la versión clásica se duplican con cada qubit que se agrega. El cociente entre el tiempo de ejecución de la simulación y el tiempo de ejecución de la versión clásica disminuye a entre 3 y 4,7. En Shor, el circuito pasa de un tiempo de ejecución de 0,06 s con 8 qubits de conteo a un tiempo de ejecución de unos 26 s con 24 qubits de conteo y a un tiempo de ejecución de unos 58 s con 25 qubits de conteo en la CPU. En cambio, la división de prueba clásica tarda entre 0,3 y 3 ms hasta 32 bits. Como los problemas son diferentes, no se comparan.

C. Memoria RAM

El pico de la RAM incluye la memoria base que utiliza el proceso (entre 330 y 484 MB, que se utiliza por Python, Qiskit y las bibliotecas). En las simulaciones por vector de estado en la CPU, el pico de la RAM es igual a la memoria base más $16 * 2^n$ bytes. La diferencia entre el pico de la RAM y la memoria base más $16 * 2^n$ bytes es menor que 4 MB. En QFT y en Shor con 29 qubits, el pico de RAM fue de 8,577 MB (381 MB de memoria base más 8,192 MiB). Con 28 qubits, el pico de la RAM fue de unos 4,480 MB. 30 qubits requerirían 16 GiB de memoria, lo que es más de los 15,34 GiB de memoria que tiene el equipo (Figura 3). En la GPU, el vector de estado se almacena en la memoria de video. En Shor, la RAM del proceso se mantuvo en unos 487

MB con 28 qubits. En QFT, la RAM del proceso llegó a 8,107 MB con n = 28. Esto es consistente con que el vector final se devuelva al anfitrión (no se ha comprobado). En Deutsch- Jozsa, la RAM del proceso se mantuvo en el valor base (333 MB) hasta que n = 32. En cambio, un vector de 33 qubits habría requerido unos 128 GiB de memoria.

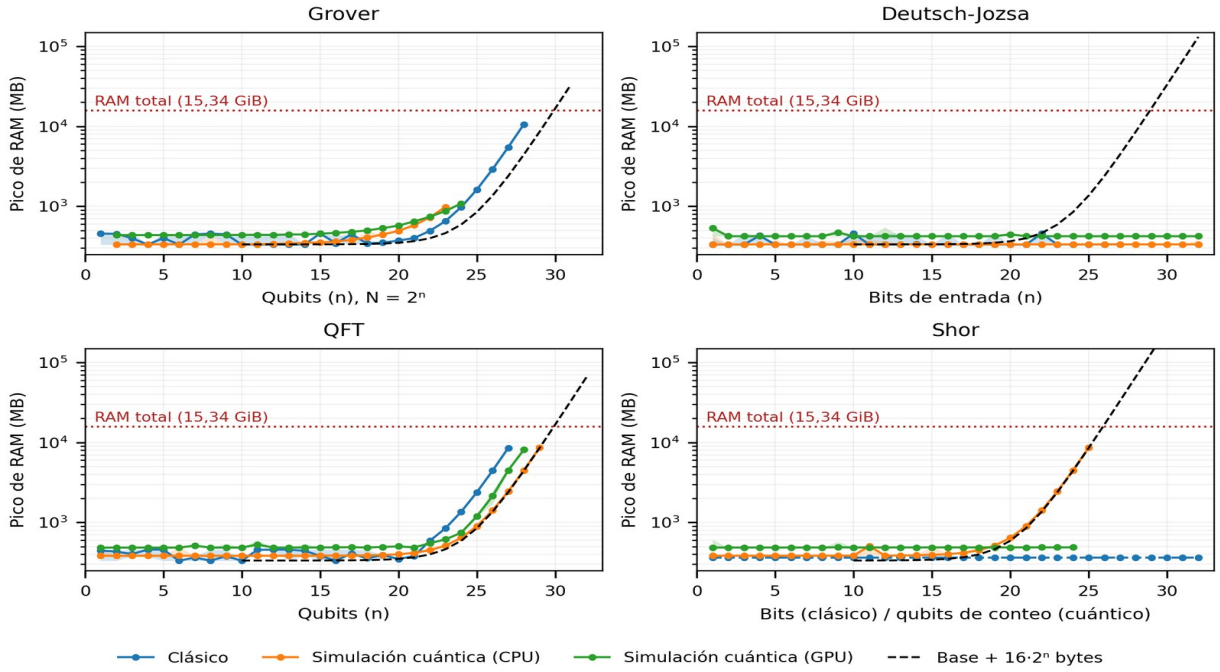


Figura 3. Pico de la RAM del proceso de cada algoritmo, versión y tamaño. El pico de la RAM fue medido por el proceso padre (se tomó una muestra cada 0,1 s). El pico de la RAM incluye la memoria base que utiliza el proceso. La línea punteada negra es igual a la memoria base más $16 * 2^n$ bytes, donde n es el número de qubits que utiliza el circuito (en Deutsch-Jozsa, n + 1; en Shor, el número de qubits de conteo + 4). El eje de las ordenadas está en escala logarítmica.

Nota. Elaboración propia.

Las versiones clásicas de los algoritmos utilizaron más RAM que la simulación en los tamaños más grandes debido a cómo se implementaron. En la búsqueda lineal, se almacena una lista de 2^n enteros de Python. Cada entero de Python ocupa unos 40 bytes, por lo que una lista de 2^n enteros de Python ocupa unos 10,604 MB con n = 28. En la FFT, se ocupó unas 4 veces la memoria que ocupaba el vector (8,525 MB con n = 27, frente a 2,432 MB de la simulación). En Grover, la RAM de la simulación (981 MB con n = 23 en CPU) es mayor que la memoria base más $16 * 2^n$ bytes. En una prueba que se realizó por separado (n = 17 a 19), se vio que la memoria adicional se utiliza durante la ejecución en Aer y que la memoria adicional aumenta con el número de instrucciones del circuito (de 49 a 95 MB, frente a vectores de 2 a 8 MB), no con 2^n.

D. Aceleración con GPU

Cuando las ejecuciones en la CPU duraron al menos 0,1 s, la GPU fue entre 1,5 y 3,1 veces más rápida que la CPU en Shor a partir de 11 qubits de conteo. En Grover, la aceleración varió entre 1,0 y 3,0 hasta que n = 18. A partir de n = 19, la aceleración creció de 1,9 (n = 19) a 3,35 (n = 23). En la transformada de Fourier, la aceleración pasó de 1,6 (n = 21) a unas 3,2 veces desde n = 26 (Figura 4). Cuando las ejecuciones duraron menos de 0,1 s, la GPU no tuvo ninguna ventaja. En Grover, el uso máximo de la GPU

fue del 100 % en las tres repeticiones desde n = 17.

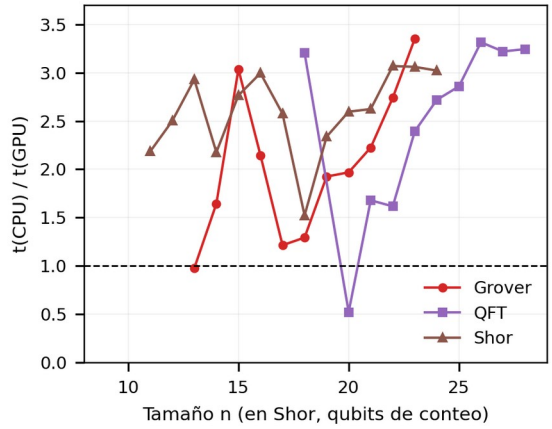


Figura 4. Aceleración de la GPU sobre la CPU (tiempo de ejecución en CPU / tiempo de ejecución en GPU, medianas de las 3 repeticiones, solo ejecuciones con t(CPU) ≥ 0,1 s).

Nota. Elaboración propia.

E. Variación entre repeticiones y límites de viabilidad

La variación del tiempo de ejecución entre repeticiones fue de 12,1 % (mediana) cuando las ejecuciones duraron menos de 0,1 s (237 condiciones). La variación del tiempo de ejecución entre repeticiones fue de 9,3 % (mediana) cuando las ejecuciones duraron entre 0,1 y 1 s (46 condiciones). La variación del tiempo de ejecución entre repeticiones fue de 5,2 % (mediana) cuando las ejecuciones duraron al menos 1 s (46 condiciones; el percentil 90 fue de 11,5 %). En las ejecuciones que duraron al menos 1 s, la GPU fue la más estable (mediana de 0,7 % en Grover y Shor) y las simulaciones en CPU variaron entre 6,4 y 10,0 %. En las ejecuciones que duraron poco tiempo, la resolución de 0,1 ms del cronómetro y las tareas del sistema tienen mucha influencia. Ninguna de las diferencias afectó a las tendencias, y los límites de las nueve series que los alcanzaron fueron los mismos en las tres repeticiones (Tabla 3). Los límites de QFT, Shor y la búsqueda lineal se deben a la falta de memoria. Los límites de Grover simulado y de Deutsch-Jozsa clásico se deben al tiempo límite de 600 s y no a un límite del equipo.

Tabla 3. Último tamaño con resultado en las tres repeticiones y causa del límite

Algoritmo	Clásica	Cuántica (CPU)	Cuántica (GPU)
Grover	28 (RAM)	23 (tiempo)	24 (tiempo)
Deutsch-Jozsa	27 (tiempo)	32 (sin fallo)	32 (sin fallo)
QFT	27 (RAM)	29 (RAM)	28 (memoria de video)
Shor	32 (sin fallo)	25 (RAM)	24 (memoria de video)

Nota. Tiempo: límite de 600 s por corrida. En QFT y Shor en CPU, Aer admite hasta 29 qubits (30 requerirían 16 GiB de vector); en GPU, 29 qubits (8 GiB) superan los 6 GB de memoria de video. En Shor, el tamaño son bits (clásica) o qubits de conteo (cuántica). Grover cuántico no pudo construirse con n = 1. Aer usó el método estabilizador, que no admite GPU.

V. Discusión

De acuerdo con los resultados, el costo de simular un algoritmo cuántico depende del método de simulación y de la implementación, y no solo del número de qubits. Con el método de vector de estado

(Grover, QFT y Shor), cada qubit adicional multiplica el tiempo de ejecución por un factor de 2 a 3,2 y duplica exactamente la memoria; el límite es de 29 qubits en CPU y 28 en GPU. Con el método estabilizador (Deutsch-Jozsa), el tiempo y la memoria se mantienen casi constantes hasta n = 32. Esto concuerda con que los circuitos estabilizadores se pueden simular de forma eficiente (Aaronson & Gottesman, 2004) y con que el método automático de Qiskit los selecciona (IBM Quantum, s. f.). Se comprobó por separado que Qiskit devuelve method = stabilizer al simular un circuito estabilizador tanto en CPU como en GPU. Como ese método no admite GPU, la serie de GPU de Deutsch-Jozsa es, en la práctica, igual a la de CPU.

En Grover, la mejora cuadrática en el número de consultas (Grover, 1996) no se traduce en un ahorro cuando se simula el circuito. En Grover, se aplican un orden de √N iteraciones. Cada iteración contiene un orden de n puertas que recorren un vector de 2^n amplitudes. Por lo tanto, el costo total de la simulación de Grover crece aproximadamente como $2^{\lceil 1.5n \rceil}$, que es unas 2,8 veces por qubit. Este resultado es del mismo orden que el de 2,6 a 3,2 que se midió. Como el tiempo de ejecución de la versión clásica solo se duplica con cada qubit que se añade, el cociente entre el tiempo de ejecución de la simulación y el tiempo de ejecución de la versión clásica aumenta de 52 a 3,221 veces. Este resultado no contradice la ventaja teórica de Grover, sino que demuestra que esa ventaja no se puede observar cuando se simula el circuito en un hardware clásico.

La memoria de la simulación por vector de estado se puede predecir y es la que limita el tamaño del circuito en este equipo. En cambio, la comparación con la versión clásica depende de cómo se implementó cada algoritmo: la búsqueda lineal (lista de enteros de Python) y la FFT (arreglos auxiliares) usaron más memoria que la simulación en los tamaños más grandes. Por tanto, la memoria de la simulación crece de forma exponencial y determinista, pero no se puede afirmar que sea mayor o menor que la de una versión clásica en general. La GPU redujo el tiempo de ejecución unas 3 veces en los tamaños más grandes, pero no cambió la forma del crecimiento, y su beneficio fue menor que las 14 veces de Faj et al. (2023) con una GPU A100. Las diferencias de hardware y de configuración entre los equipos impiden que se pueda atribuir esta diferencia a una sola causa. Además, restó un qubit de alcance en QFT y Shor porque tiene menos memoria (6 GB) que la RAM del equipo, y amplió uno en Grover, donde el límite era de tiempo.

Los resultados corresponden a un solo equipo, a la versión 0.15.1 de Qiskit Aer y a tres repeticiones. El tiempo límite de 600 s restringe el tamaño de Grover simulado y de Deutsch-Jozsa clásico. Las líneas base de Grover y de Deutsch-Jozsa están escritas en Python interpretado, por lo que una implementación en un lenguaje compilado ampliaría la diferencia entre el tiempo de la simulación y el de la versión clásica. Faltan mediciones válidas de CPU, disco y memoria de la GPU (previstas en el diseño inicial), y no se verificó la corrección de los resultados de Grover, Deutsch-Jozsa ni QFT. Los resultados no se extienden a otros métodos de simulación, como el producto de estados matriciales, ni a hardware cuántico real.

Entre las fortalezas: cada medición se hizo en un proceso aislado con vigilancia de recursos, se conservaron los datos crudos y los fallos se reportan como límites de viabilidad.

VI. Conclusiones

En respuesta a la pregunta de investigación, si bien el uso de recursos por parte de la simulación cuántica se incrementa

exponencialmente con el tamaño, la relación con la versión clásica depende del algoritmo y del método de simulación. En el caso del vector de estado, el tiempo requerido fue entre 3 y 3,221 veces mayor que el clásico en Grover y en la transformada de Fourier, y la memoria se mantuvo en $16 * 2^n$ bytes más una base fija, lo que limitó el alcance a 29 qubits en CPU y 28 en GPU. Deutsch-Jozsa, simulado con el método estabilizador, fue más rápido que su versión clásica desde los 12 bits, con tiempo y memoria casi constantes.

Para quienes diseñan experimentos en Qiskit Aer con hardware estándar, los resultados muestran que el número de qubits alcanzable está limitado por la memoria (RAM o de video) y que se debe elegir el método de simulación en función de la estructura del circuito. La GPU acelera aproximadamente 3 veces en tamaños grandes y no ofrece ventajas en los pequeños, y la variabilidad entre repeticiones (mediana de 5 % en corridas de 1 s o más) no afectó las tendencias.

Los futuros estudios deben medir la carga de CPU y disco por proceso ademas de registrar la memoria de video y la corrección de cada corrida; fijar el método de simulación del Deutsch-Jozsa para que sea comparable con los otros; utilizar líneas base clásicas optimizadas; aumentar las repeticiones; y probar otros métodos de simulación.

VII. Declaraciones

A. Contribución del autor (CrediT)

J.T.M.S.: conceptualización, metodología, investigación, curación de datos, análisis formal, visualización, redacción y revisión del manuscrito.

B. Financiación

El autor declara que este trabajo no contó con financiación externa.

C. Conflicto de intereses

El autor declara que no existe ningún conflicto de intereses relacionado con la publicación de este artículo.

D. Disponibilidad de datos

Los datos crudos de las 1.032 corridas (corridas_crudas.jsonl) y los scripts de medición y análisis están disponibles a solicitud del autor.

E. Uso de herramientas de inteligencia artificial

Se utilizó *Elicit* para la búsqueda y síntesis de literatura que sirvió de base para el marco teórico (sección II). *Claude* (Anthropic) se empleó como apoyo en la estructura y unificación del documento y su adaptación a esta plantilla, en la homogeneización de citas y referencias (APA, 7.ª edición), en la escritura de los scripts de ejecución de las mediciones (ejecutar_todo.py) y de verificaciones complementarias (verificaciones.py) y en el análisis de los datos crudos. El autor revisó el contenido producido con apoyo de estas herramientas y asume la responsabilidad por el documento final.

VIII. Referencias

[1] Aaronson, S., & Gottesman, D. (2004). Improved simulation of stabilizer circuits. Physical Review A, 70, 052328. https://doi.org/10.1103/PhysRevA.70.052328

[2] Cerrudo-Herrera, J., Talaván-Vega, D., Rodríguez-Oliver, P., et al. (2026). Benchmarking quantum computing statevector simulators on high-performance computing. SIMULATION: Transactions of the Society for Modeling and Simulation International. Advance online publication. https://doi.org/10.1177/00375497261438515

[3] Díaz, G., Steffenel, L., & Barrios Hernández, C. (2021). On the resource consumption of software quantum computing simulators. DYNA, 88(218). https://doi.org/10.15446/dyna.v88n218.90781

[4] Faj, J., Peng, I. B., Wahlgren, J., & Markidis, S. (2023). Quantum computer simulations at warp speed: Assessing the impact of GPU acceleration: A case study with IBM Qiskit Aer, Nvidia Thrust & cuQuantum. En 2023 IEEE International Conference on e-Science (pp. 1–10). https://doi.org/10.1109/e-Science58273.2023.10254803

[5] Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. En Proceedings of the 28th Annual ACM Symposium on Theory of Computing (pp. 212–219). https://doi.org/10.1145/237814.237866

[6] Häner, T., & Steiger, D. S. (2017). 0.5 petabyte simulation of a 45-qubit quantum circuit. En Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. https://doi.org/10.1145/3126908.3126947

[7] IBM Quantum. (s. f.). AerSimulator — Qiskit Aer documentation. https://qiskit.github.io/qiskit-aer/stubs/qiskit_aer.AerSimulator

[8]Ma, N., & Li, H. (2024). Understanding and estimating the execution time of quantum circuits [Preprint]. arXiv. https://arxiv.org/abs/2411.15631

[9] Markov, I. L., & Shi, Y. (2008). Simulating quantum computation by contracting tensor networks. SIAM Journal on Computing, 38(3), 963–981. https://doi.org/10.1137/050644756

[10] Soltaninia, M., & Zhan, J. (2023). Comparison of quantum simulators for variational quantum search: A benchmark study. https://doi.org/10.48550/arXiv.2309.05924

[11] Strano, D., Bollay, B., Blaauw, A.,Shammah, N., Zeng, W. J., & Mari, A. (2023). Exact and approximate simulation of large quantum circuits on a single GPU [Preprint]. arXiv. https://arxiv.org/abs/2304.14969

[12] Tsuji, M., & Sato, M. (2022). Performance analysis of a state vector quantum circuit simulation on A64FX processor. https://doi.org/10.1109/CLUSTER51413.2022.00074

[13] Vallero, M., Vella, F., & Rech, P. (2025). State of practice: Evaluating GPU performance of state vector and tensor network methods. Future Generation Computer Systems, 164, 107927. https://doi.org/10.1016/j.future.2025.107927

[14] Vidal, G. (2003). Efficient classical simulation of slightly entangled quantum computations. Physical Review Letters, 91(14), 147902. https://doi.org/10.1103/PhysRevLett.91.147902

[15] Simulations of quantum algorithms using the Qiskit Aer library and the matrix product state method on CPU and GPU. (2025). [Registro consultado sin autoría ni datos completos de publicación.] https://tit.duikt.edu.ua/telecommunication/article/download/2635/2505

[16] Zhao, Y.-L., Chen, Y., Li, H., Wang, Y., Chang, K.-Y., Wang, B., Li, B., & Han, Y.-H. (2023). Full state quantum circuit simulation beyond memory limit. https://doi.org/10.1109/ICCAD57390.2023.10323666